# Solution & Workflow

## The problem

Pinned, aging dependencies quietly break projects or carry known advisories. In the LinkWatch starter repository, `Jinja2==2.11.3` no longer imports against a current MarkupSafe, so one of its ten tests fails, and the package index lists 8 advisories for that exact version. Developers need to know **which** dependencies are at risk, **where** they are used, **what to change**, and **proof** that the change works before it touches their repository.

## Our solution

DepGuard AI analyses a real repository and explains each at-risk dependency with cited evidence. The user chooses **Repair**, **Ignore** or **Review later** for each finding. A **local Gemma 4** model proposes a concrete patch; DepGuard validates it and tests it against a baseline in an isolated copy. The user reviews the diff and the results, and only after explicit approval is anything applied, to a working copy and never to the analysed repository. Everything ends in an evidence-backed **Dependency Exit Capsule**.

## Workflow

1 Repository > 2 Findings > 3 Ask Gemma 4 > 4 Repair > 5 Verify > 6 Publish > 7 Reports & capsule

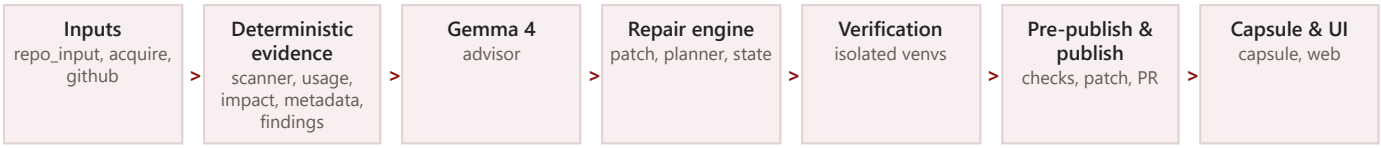
| Step | What happens | What the user gets |
|---|---|---|
| **1 · Repository** | Open a **ZIP**, a **local Git repository** or a **GitHub** repository; ZIPs are validated before extraction. | Active-repository card with branch, exact commit and dirty state. |
| **2 · Findings** | Deterministic scan, usage and impact analysis, evidence-based severity. | One finding per at-risk dependency, with file:line usage and Repair / Ignore / Review later. |
| **3 · Ask Gemma 4** | The local model advises on **every** at-risk finding, in small batches. | Schema-validated advice, or "no reliable recommendation"; always labelled unverified. |
| **4 · Repair** | **Mode A:** the user triggers one proposal. **Mode B:** the user enables automatic *preparation* for chosen findings, with bounded retries. | Patches with per-file diff, reason, origin and evidence. Nothing is applied. |
| **5 · Verify** | Baseline and patched tests run in separate temporary copies, each with its own virtual environment. | Exact commands, exit codes, durations, environment and failure kind. |
| **6 · Publish** | Pre-publish checks, then a local patch export or a GitHub branch and pull request with **two separate approvals**. | A `git apply` patch, or a new branch and PR. Never a merge or force-push. |
| **7 · Reports & capsule** | All evidence is assembled and validated against a JSON schema. | Dependency Exit Capsule (JSON, Markdown, HTML): combined, plus one per dependency. |

> **The user stays in control.** Nothing is applied automatically in either repair mode. Applying needs the typed word `APPLY`; publishing needs `PUBLISH` and then `OPEN PR`. Ignored findings stay in the reports together with the reason.

> **Demonstrated on LinkWatch (a real run).** 11 findings (4 critical, 2 medium, 5 low) → Gemma 4 advised on all 6 at-risk ones → a Mode A repair relaxed the Jinja2 pin to `Jinja2>=3.1.3` in both manifests → tests in isolation went from **9 passed, 1 failed** to **10 passed, 0 failed** → applied only to a working copy; the original manifests are unchanged.

# Technical Approach

Deterministic evidence first, a local model second, and executed tests as the judge.




## Deterministic evidence

- **Manifests:** `requirements*.txt` and `pyproject.toml`, parsed with `packaging` and `tomllib`. Pipfile, Poetry/uv/PDM lockfiles and `setup.py` are detected and reported as unsupported, never guessed.
- **Usage:** AST-based import mapping with file:line. Dynamic imports are reported, not resolved. A dependency → module → test impact graph lists its own uncertainty.
- **Severity:** documented rules; for example *critical* means the package index lists an advisory for the exact version in use. "No advisory listed" never means "secure".
- **Package-index data** (advisories, yanked flag, Requires-Python, license) is fetched only on request and cached with its source URL and fetch time. Offline it reads "not checked".

## Verification and patch safety

- Baseline and patched code each run in their own temporary copy and virtual environment. Every step records its command, exit code, duration and timeout.
- Results are classified *verified*, *partially verified*, *failed*, *blocked* or *not run*, never optimistically.
- Every edit is validated: only manifests and source files that use the dependency, no deletions or new files, Python must still parse, TOML must stay valid, process/network/eval calls are refused, at most 12 edits per patch.
- Line endings are preserved; a patch becomes **stale** if its files change; applying needs `APPLY` and takes a snapshot for rollback.

## How Gemma 4 is used

- **Runs locally:** `gemma4:e2b` (4.6B parameters, Q4_K_M, 131,072-token context) through Ollama 0.40.4 at `127.0.0.1:11434`. No hosted API, no paid service, no cloud.
- **Structured and deterministic-leaning:** called through the Ollama chat API with a JSON schema and temperature 0. Output is schema-validated; unusable output is rejected or filtered (for example, file names offered as package candidates).
- **Given evidence, not just a name:** version in use, usage sites, advisories and baseline test errors. It is told never to contradict the evidence and to answer "no reliable recommendation" when evidence is insufficient.
- **Two roles:** advice for every at-risk dependency (batches of two), and one repair proposal per finding as exact find/replace edits with assumptions, concerns and suggested tests. A failed test's output is fed back for a bounded revision (1 to 5 attempts, default 2).
- **Repository text is data:** the prompts tell the model to treat files, comments, READMEs and package descriptions as data, never as instructions. This is a safeguard, not a guarantee, which is why every edit is validated and then tested.

## Security

- Dashboard binds `127.0.0.1`, rejects foreign `Host` / `Origin` headers and POSTs without `X-DepGuard`; no shell or arbitrary-command endpoint; paths limited to authorised roots.
- ZIPs are checked for traversal, absolute or reserved names, symlinks, size, entry count and compression ratio before extraction.
- GitHub tokens stay in memory only and are redacted from logs and reports.

**Stack and quality.** Python 3.12 / 3.13; standard-library `http.server`; plain HTML, CSS and JavaScript with no build step; dependencies are only `jsonschema`, `packaging` and `pytest`. **366 automated tests** (mocked AI, GitHub and verifier) pass on Python 3.12.10 and 3.13.14, plus one live Gemma 4 test.

# Feasibility & Viability

## Feasible: it already works end to end

- The full workflow was exercised on a real repository (LinkWatch) through the real dashboard: ZIP, local Git and GitHub-download inputs, both repair modes, verification, patch export and capsules.
- **Runs on a student laptop** (15 GB RAM). Measured: about 21 tokens/s with the model on the CPU, and about 38 tokens/s on the laptop's RTX 3050 (4 GB) once Ollama detects it. A full analysis takes minutes, not seconds.
- **Zero running cost:** no paid API and no cloud; three small Python dependencies.
- **Quality evidence:** 366 tests on two Python versions, a live model test, and schema validation of every capsule.

## Viable: why it can be adopted and maintained

- **Privacy:** code and prompts never leave the machine, which suits private repositories. The only network use is what you choose: package-index lookups (package names and versions), installs during verification, and GitHub.
- **Small maintenance surface:** standard-library server, no front-end build, minimal dependencies, and a capsule schema that is extended additively.
- **Open source:** MIT licensed, unchanged from the starter. Gemma 4 weights are not redistributed; users pull the model themselves.
- **Fits existing workflows:** read-only analysis, `git apply`-compatible patches, and an optional pull request to a *new* branch only after two approvals.
- **Complements existing tools:** scanners and update bots find and bump; DepGuard supports the decision and the proof.

## Risks and how each is handled today

| Risk | Mitigation that exists in the current build |
|---|---|
| A small local model can be wrong or weak | Output is schema-validated and never trusted: edits are validated, then tested. An advisory-coverage check exposes fixes that still allow affected releases. "No reliable recommendation" is an accepted answer. |
| Advisory data is incomplete or unavailable | Index lookup is opt-in and cached with source and time; offline it reads "not checked". Data is never presented as an audit. |
| Running a repository's tests executes its code | Isolated virtual environment in a temporary copy with timeouts; ZIP and GitHub inputs need a separate explicit confirmation. Disclosed plainly: a virtual environment is **not** a sandbox. |
| Passing tests do not prove compatibility | Partial results (skipped or fewer tests) are labelled partial; the comparison and the pre-publish review state that passing tests do not prove full compatibility. |
| GitHub publication was not run live | Covered by mocked tests; two typed approvals; never merges or force-pushes. The local patch export is the path exercised for real. |

## Licensing note

The license text bundled with the local model says Apache 2.0. This was **not cross-checked** against Google's official terms.

> **Next steps (roadmap, not implemented):** container sandbox with CPU and memory limits; lockfile support (Poetry, uv, pip-tools); a live GitHub pull-request test; larger Gemma 4 variants where hardware allows; other ecosystems beyond Python.

# Impacts & Benefits

## Who benefits

- **Maintainers** of aging Python projects who must decide whether to upgrade, replace or remove a dependency.
- **Students and first-time contributors** who want to modernise legacy code safely, with an explanation of every finding.
- **Small teams** without a dedicated security engineer.
- **Reviewers**, who receive the diff, the reasons and the test evidence in one place.

## Demonstrated impact (measured on the real LinkWatch run)

| What happened | Result |
|---|---|
| Dependencies analysed | 26 declared dependencies, 11 findings: 4 critical, 0 high, 2 medium, 5 low. |
| Critical findings | Package-index advisories for the exact versions in use: jinja2 2.11.3 (8), requests 2.31.0 (6), idna 2.7 (4), setuptools 75.8.0 (4), each with source URL and retrieval time. |
| Root cause of a failing test | Baseline 9 passed, 1 failed: the `soft_unicode` ImportError from Jinja2 2.11.3 with a current MarkupSafe. |
| Verified repair candidate | Relaxing the Jinja2 pin in both manifests: **10 passed, 0 failed** in isolation, no regressions. Applied only to a working copy; the original manifests still say 2.11.3. |
| A bad AI suggestion caught | A combined Gemma proposal that relaxed `setuptools` **failed verification** (pytest exit 4) and was recorded as a failure instead of being presented as a fix. |

## Benefits

- **Better decisions.** Each finding carries severity, evidence with source and retrieval time, usage locations and the modules and tests affected. Repair / Ignore / Review later keeps a clear record.
- **Safer changes.** Verification happens before anything is applied; edits are validated; nothing is applied automatically; rollback and stale-patch detection exist.
- **Privacy and cost.** Local inference means code never leaves the machine and there is nothing to pay for.
- **Transparency and trust.** Observed facts, package-index data, AI claims and executed test results are labelled separately, and the capsule records the exact commands, exit codes and durations a reviewer can audit.

For open source, this turns dependency upkeep into a contribution a newcomer can attempt safely: the patch arrives with its reasons and its test evidence.

> **What we deliberately do not claim.** DepGuard is not a security audit, and passing tests are not proof of full compatibility. It does not replace vulnerability scanners. **Not measured yet:** maintainer time saved, and the accuracy of Gemma's advice across many repositories (only LinkWatch was exercised for real).

# Research & References

## Research questions and what we found while building it

| Question | Finding |
|---|---|
| **1.** Can a small local model give useful dependency advice? | Yes, with guardrails. In an earlier run, without the collected evidence, Gemma 4 said "keep requests" even though the index lists 6 advisories for the pinned version; once the findings were included in the prompt it advised on all 6 at-risk dependencies. It also offered file names as "replacement candidates" (now filtered) and proposed `Jinja2>=3.1.3`, a lower bound that still allows versions affected by advisories the index lists as fixed only in 3.1.4 to 3.1.6. An advisory-coverage check was added to expose such weak fixes (unit-tested against the real advisory data; not re-run live). |
| **2.** What should count as proof that a change is safe? | The project's own tests, run baseline versus patched in separate isolated environments with exact commands and exit codes. Even then it is only as strong as the tests, so results say so. The verifier caught a failing combined proposal (pytest exit 4), and it is recorded as a failure. |
| **3.** Which data can be trusted, and how should it be shown? | Keep four kinds of statements apart: observed, package-index metadata, AI-generated, and executed tests. "No advisory listed" is not "secure", and offline data is "not checked". |

## Related tools

| Tool | What it does (from its public documentation) | How DepGuard relates |
|---|---|---|
| **pip-audit [14]** | Scans Python environments and requirements for packages with known vulnerabilities. | Complementary. DepGuard reads advisories the index lists for the exact version, then adds a recommendation and verification. |
| **OSV-Scanner [15]** | Finds known vulnerabilities in a project's dependencies using the OSV database. | A complementary data source; DepGuard does not query OSV directly. |
| **Dependabot [16], Renovate [17]** | Open automated pull requests that update dependencies. | They propose version bumps. DepGuard supports the upgrade / replace / remove decision and attaches test evidence. |

## References

Every link below was checked and returned HTTP 200 on 15 October 2026 (some redirect to their current location).


1. PEP 440, Version Identification and Dependency Specification
   `peps.python.org/pep-0440/`
2. PEP 508, Dependency specification for Python Software Distributions
   `peps.python.org/pep-0508/`
3. PEP 621, Storing project metadata in pyproject.toml
   `peps.python.org/pep-0621/`
4. Python docs: venv
   `docs.python.org/3/library/venv.html`
5. Python docs: ast
   `docs.python.org/3/library/ast.html`
6. Python docs: tomllib
   `docs.python.org/3/library/tomllib.html`
7. PyPI JSON API
   `docs.pypi.org/api/json/`
8. packaging (PyPA)
   `packaging.pypa.io`
9. pytest
   `docs.pytest.org`
10. JSON Schema, and the jsonschema library for Python
    `json-schema.org`
    `python-jsonschema.readthedocs.io`
11. Gemma documentation, Google AI for Developers
    `ai.google.dev/gemma/docs`
12. Gemma 4 on Ollama
    `ollama.com/library/gemma4`
13. Ollama API reference
    `github.com/ollama/ollama/blob/main/docs/api.md`
14. pip-audit (PyPA)
    `github.com/pypa/pip-audit`
15. OSV-Scanner
    `google.github.io/osv-scanner`
16. GitHub Dependabot
    `docs.github.com/en/code-security/dependabot`
17. Renovate
    `docs.renovatebot.com`
18. LinkWatch starter repository (the demonstration target)
    `github.com/rishrek/LinkWatch`


> **Evidence in the repository.** The real-run section of `README.md`, the recorded capsules in `demo-output/` (JSON, Markdown, HTML), and the 366-test suite under `depguard/tests/`.